Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 02

Diseño y Desarrollo de Simuladores y Videojuegos

DOCENTE:

Bestard Aroche, Yunior

CURSO:

Testing de Videojuegos

6 - C26 - Sección A

TESTING DE VIDEOJUEGOS: DEFINICIÓN CONCEPTOS Y TIPOS DE TESTING

I. Capacidades

- **Diseña casos de prueba unitarios** identificando comportamiento, estado inicial, entrada y resultado esperado a partir de métodos reales de su prototipo de tesis.
- **Implementa pruebas automatizadas** en Unity Test Framework aplicando la estructura Arrange–Act–Assert y utilizando al menos tres tipos de aserción distintos.
- **Ejecuta y analiza resultados de pruebas** en Test Runner, distinguiendo una prueba aprobada de una fallida y justificando qué revela cada una sobre la calidad de su prototipo.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VII. Procedimiento

1. Ejercicio 1 (3:40 – 3:55 p.m., 15 min) — Inventario de métodos testeables

Revise su prototipo y liste 5 métodos o comportamientos candidatos a prueba unitaria (cálculos, reglas de estado, validaciones). Justifique cada elección con un criterio de la teoría (entrada clara, resultado verificable, regla determinística, etc.).

a. Registrar un error o intento fallido

Es testeable porque tiene una entrada clara (registrar un error), una regla determinística (incrementar el contador) y un resultado verificable (el número de intentos aumenta en 1).

b. Reducir la integridad del sistema ante un error

Permite comprobar un cálculo determinístico: a partir de un valor de integridad y una penalización definida se obtiene un nuevo valor. El resultado puede compararse directamente con el valor esperado.

c. Determinar si una opción es segura

Presenta una entrada claramente definida (la opción analizada) y un resultado booleano verificable (true o false). La regla de seguridad es determinística, por lo que la misma entrada debe producir siempre el mismo resultado.

d. Determinar si el acceso debe ser rechazado por exceso de errores

Tiene una regla de estado clara: mientras los errores estén por debajo del límite se permite continuar; al alcanzar el límite se rechaza el acceso. Es fácilmente verificable mediante valores concretos.

e. Avanzar al siguiente nivel del reto

Es una regla de estado determinística: dado un nivel actual válido y una condición de superación, el sistema debe producir un nivel siguiente concreto. Permite verificar que la progresión ocurra correctamente y que no se produzcan niveles fuera del rango establecido.

2. Ejercicio 2 (3:55 – 4:15 p.m., 20 min) — Diseño de casos de prueba
Para cada uno de los 5 métodos, complete una tabla con: Comportamiento / Estado inicial / Entrada / Acción / Resultado esperado.

a. Registrar un error o intento fallido

Estado inicial: El jugador tiene 2 errores registrados.

Entrada: Registrar un nuevo error.

Acción: Ejecutar el método encargado de registrar el error.

Resultado: El contador de errores aumenta de 2 a 3.

b. Reducir la integridad del sistema ante un error

Estado inicial: La integridad de la sesión es de 100% y la penalización definida por el sistema es de 15%.

Entrada: Se produce un error durante la verificación.

Acción: Ejecutar el método que aplica la penalización de integridad.

Resultado: La integridad disminuye de 100% a 75%.

c. Determinar si una opción es segura

Estado inicial: El reto presenta una opción identificada como segura.

Entrada: Seleccionar o analizar la opción segura.

Acción: Ejecutar la validación de seguridad de la opción.

Resultado: El sistema determina que la opción es segura y devuelve true.

d. Determinar si el acceso debe ser rechazado por exceso de errores

Estado inicial: El jugador tiene 2 errores y el límite establecido es de 3 errores.

Entrada: Registrar un nuevo error.

Acción: Ejecutar la regla de validación del límite de errores.

Resultado: El contador llega a 3 y el sistema determina que el acceso debe ser rechazado.

e. Avanzar al siguiente nivel del reto

Estado inicial: El jugador se encuentra en el nivel 2 y ha superado correctamente el reto.

Entrada: Confirmar la superación del nivel actual.

Acción: Ejecutar el método de progresión del nivel.

Resultado: El nivel aumenta de 2 a 3 y el jugador continúa con el siguiente nivel.

3. Ejercicio 3 (4:15 – 4:30 p.m., 15 min) — Casos límite
Para al menos 2 de los métodos anteriores, defina un caso límite (valor exacto en el borde) y una entrada inválida, indicando qué debería hacer el sistema en cada caso.

Método: Registrar un error o intento fallido

Caso límite: El jugador tiene 2 errores y el máximo permitido es 3.

Comportamiento esperado: El contador pasa exactamente a 3. El sistema reconoce que se alcanzó el límite y debe rechazar el acceso.

Entrada inválida: El contador de errores se encuentra en 3, que corresponde al máximo permitido.

Comportamiento esperado: El sistema debe impedir que el contador continúe aumentando por encima del límite o rechazar la operación, según la implementación definida.

Método: Reducir la integridad del sistema ante un error

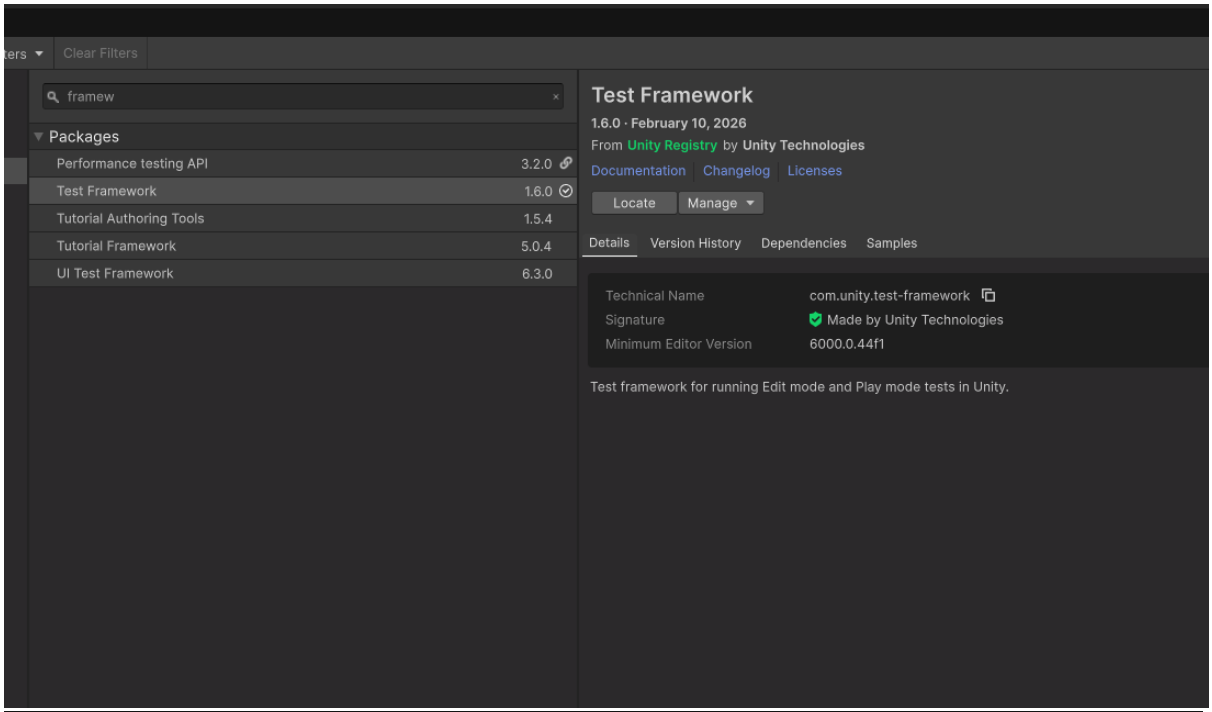
Caso límite: La integridad de la sesión es 100% y una penalización equivale a 15%.

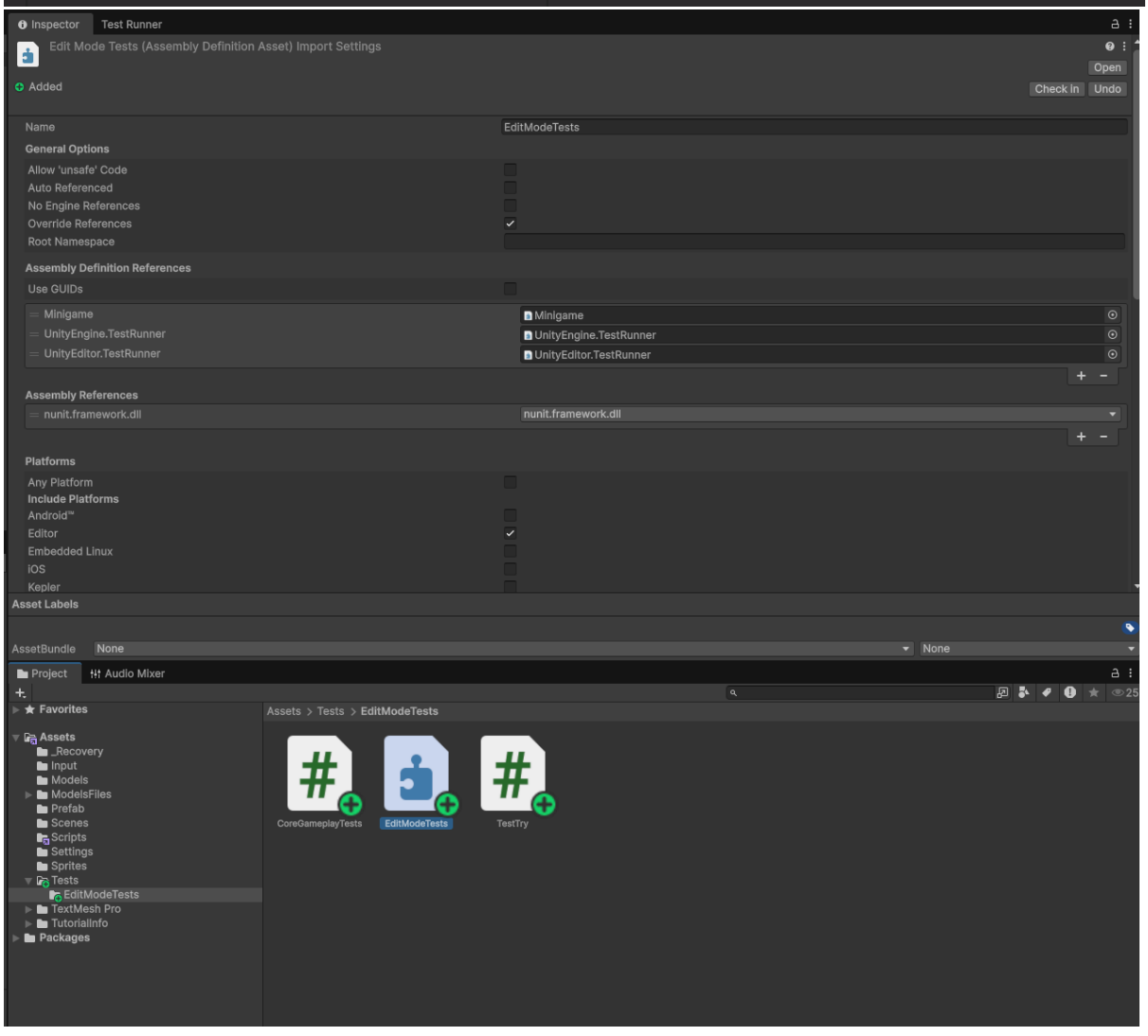
Comportamiento esperado: La integridad pasa exactamente a 0%, sin quedar por debajo de cero.

Entrada inválida: La integridad de la sesión ya se encuentra en 0%.

Comportamiento esperado: El sistema debe rechazar la operación o mantener la integridad en 0%, evitando valores negativos.

4. Ejercicio 4 (4:30 – 4:40 p.m., 10 min) — Preparación del entorno
Verifique o instale el paquete Unity Test Framework y cree un Test Assembly en Edit Mode dentro de su proyecto.


Clear Filters
framew
Packages
Performance testing API 3.2.0
Test Framework 1.6.0
Tutorial Authoring Tools 1.5.4
Tutorial Framework 5.0.4
UI Test Framework 6.3.0
Test Framework
1.6.0 · February 10, 2026
From Unity Registry by Unity Technologies
Documentation Changelog Licenses
Locate Manage
Details Version History Dependencies Samples
Technical Name com.unity.test-framework
Signature Made by Unity Technologies
Minimum Editor Version 6000.0.44f1
Test framework for running Edit mode and Play mode tests in Unity.


Inspector
Test Runner
Edit Mode Tests (Assembly Definition Asset) Import Settings
Added
Open
Check In
Undo
Name EditModeTests
General Options
Allow 'unsafe' Code
Auto Referenced
No Engine References
Override References
Root Namespace
Assembly Definition References
Use GUIDs
Minigame
UnityEngine.TestRunner
UnityEditor.TestRunner
Assembly References
nunit.framework.dll
Platforms
Any Platform
Include Platforms
Android
Editor
Embedded Linux
iOS
Kepler
Asset Labels
AssetBundle None None
Project
Audio Mixer
Favorites
Assets
_Recovery
Input
Models
ModelsFiles
Prefab
Scenes
Scripts
Settings
Sprites
Tests
EditModeTests
TextMesh Pro
TutorialInfo
Packages
Assets > Tests > EditModeTests
CoreGameplayTests
EditModeTests
TestTry

5. Ejercicio 5 (4:40 – 5:00 p.m., 20 min) — Primera prueba unitaria
Implemente la prueba del método más sencillo de su lista, aplicando explícitamente los comentarios // Arrange, // Act, // Assert.

```
using NUnit.Framework;
using UnityEngine;

0 referencias
public class FirstUnitTest
{
    [Test]
    0 referencias
    public void IsSafe_ReturnsTrue_ForACorrectDoor()
    {
        // Arrange: preparar el estado inicial del sujeto de la prueba.
        var door = new GameObject().AddComponent<DoorOption>();
        door.Initialize(DoorType.VerySafe, true, null);

        // Act: ejecutar la operación que se quiere verificar.
        bool isSafe = door.IsSafe();

        // Assert: comprobar que el resultado coincide con lo esperado.
        Assert.IsTrue(isSafe, "Una opción correcta debe considerarse segura.");

        Object.DestroyImmediate(door.gameObject);
    }
}
```

6. Ejercicio 6 (5:00 – 5:15 p.m., 15 min) — Aserción de igualdad
Implemente una prueba que use Assert.That(actual, Is.EqualTo(esperado)) sobre un método de cálculo (daño, puntaje, economía, etc.).

```
public class EqualityAssertionTest
{
    [Test]
    0 referencias
    public void TakeDamage_IntegrityEqualsExpected_AfterOneMistake()
    {
        // Arrange
        var system = new GameObject().AddComponent<IntegritySystem>();
        system.ResetIntegrity();
        float expected = system.MaxIntegrity - 25f; // penalización configurada en IntegritySystem.

        // Act
        system.TakeDamage();

        // Assert
        Assert.That(system.CurrentIntegrity, Is.EqualTo(expected));

        Object.DestroyImmediate(system.gameObject);
    }
}
```

7. Ejercicio 7 (5:15 – 5:30 p.m., 15 min) — Aserción booleana o de colección
Implemente una prueba que use Is.True, Is.False o Does.Contain sobre una regla de estado o un inventario/lista.

```
public class BooleanAssertionTest
{
    [Test]
    0 referencias
    public void HasReachedMaxStrikes_IsFalse_BelowTheLimit()
    {
        // Arrange
        var manager = new GameObject().AddComponent<StrikeManager>();
        manager.AddStrike();
        manager.AddStrike();

        // Act
        bool reached = manager.HasReachedMaxStrikes();

        // Assert
        Assert.That(reached, Is.False, "Con 2 errores de 3, el acceso aún no debe rechazarse.");

        Object.DestroyImmediate(manager.gameObject);
    }

    [Test]
    0 referencias
    public void HasReachedMaxStrikes_IsTrue_AtTheLimit()
    {
        // Arrange
        var manager = new GameObject().AddComponent<StrikeManager>();
        manager.AddStrike();
        manager.AddStrike();
        manager.AddStrike();

        // Act
        bool reached = manager.HasReachedMaxStrikes();

        // Assert
        Assert.That(reached, Is.True, "Al llegar al límite de 3 errores, el acceso debe rechazarse.");

        Object.DestroyImmediate(manager.gameObject);
    }
}
```

2559 x 1388

8. Ejercicio 8 (5:30 – 5:45 p.m., 15 min) — Aserción de excepción
Implemente una prueba con Assert.Throws<...>() para un método que deba rechazar una entrada inválida (ej. daño negativo, capacidad excedida).

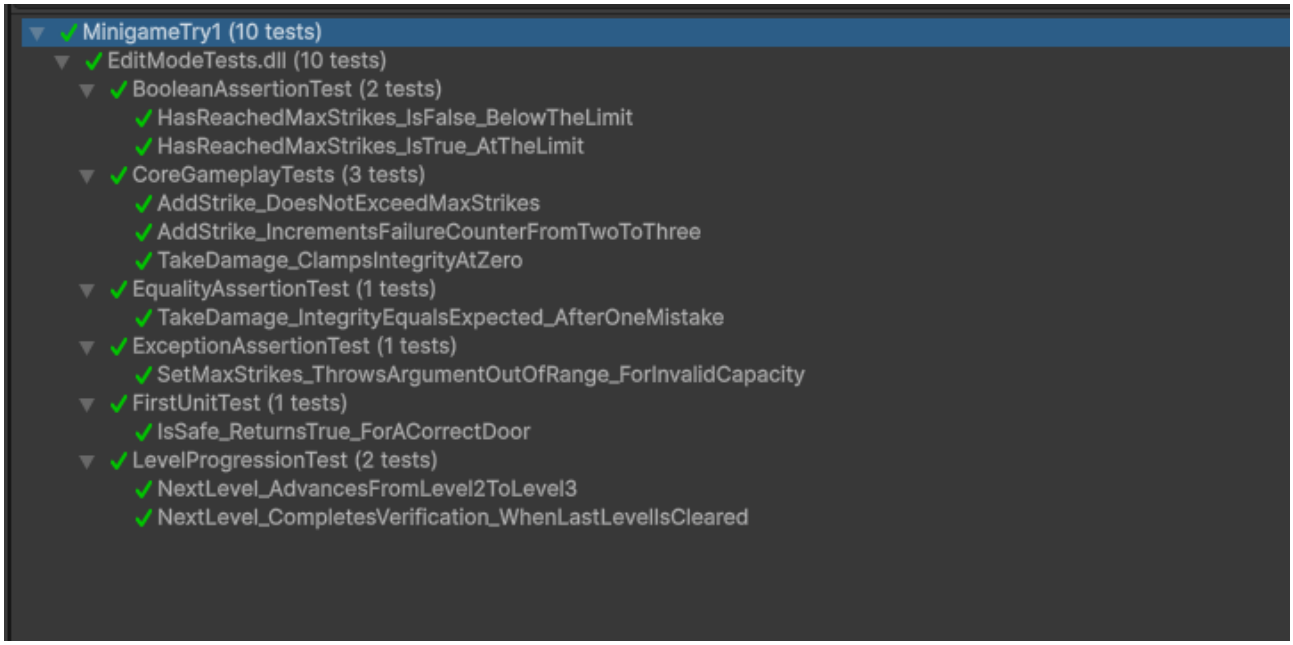
```
public class ExceptionAssertionTest
{
    [Test]
    0 referencias
    public void SetMaxStrikes_ThrowsArgumentOutOfRange_ForInvalidCapacity()
    {
        // Arrange
        var manager = new GameObject().AddComponent<StrikeManager>();

        // Act & Assert
        Assert.Throws<System.ArgumentOutOfRangeException>(() => manager.SetMaxStrikes(0));
        Assert.Throws<System.ArgumentOutOfRangeException>(() => manager.SetMaxStrikes(-5));

        Object.DestroyImmediate(manager.gameObject);
    }
}
```

2559 x 1388

9. Ejercicio 9 (5:45 – 5:55 p.m., 10 min) — Ejecución y lectura de resultados
Ejecute las 5 pruebas en Test Runner. Documente con capturas cuáles pasan y cuáles fallan, y explique la causa de cada fallo (bug real vs. expectativa mal definida).



Todas funcionaron.

10. Ejercicio 10 (5:55 – 6:00 p.m., 5 min) — Explicación en video
Grabe un video breve mostrando el código de sus 5 pruebas, su ejecución en Test Runner y una explicación de qué comprueba cada una, según lo pedido en la guía. (La grabación se completa fuera del horario de aula).

Link

https://youtu.be/-SZQ5BtujmA

Conclusiones: